Nonparametric Curve Estimation HW4

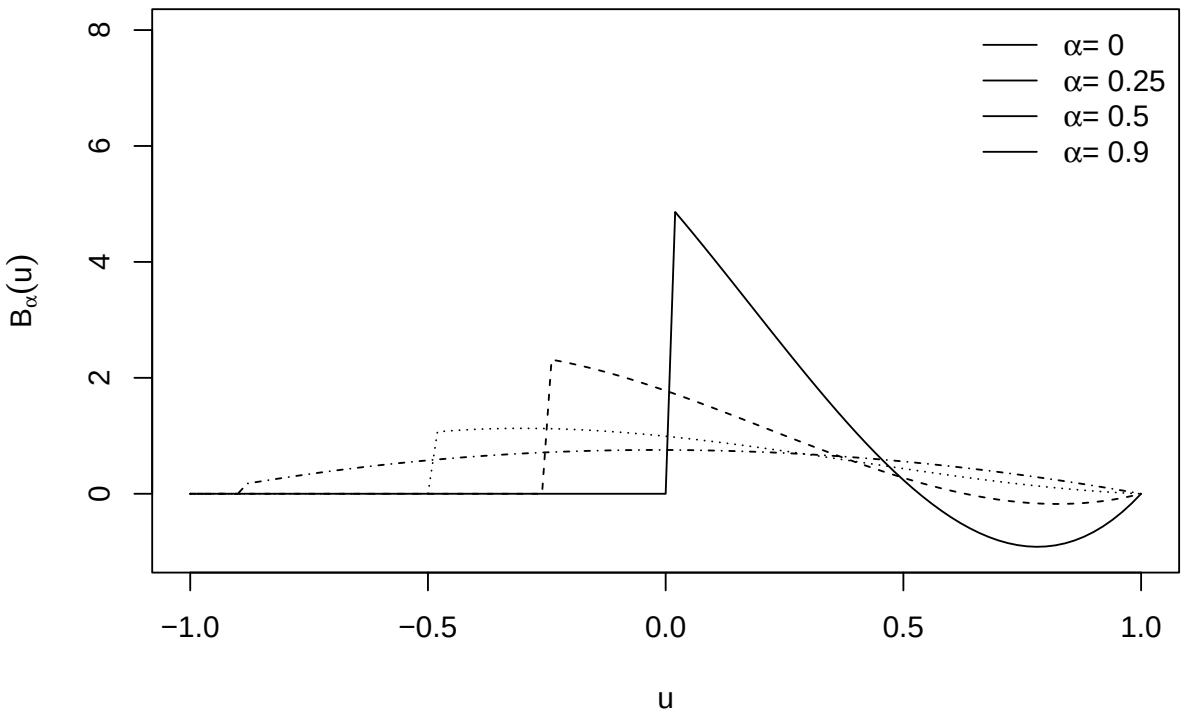
Name: 莊明儒 Student ID: 111024513

14.

Let $a_l(\alpha) = \int_{-\alpha}^{1} u^l K(u)du$. The left boundary kernel function is given by

$$B_\alpha(u) = \frac{(a_2(\alpha) - a_1(\alpha)u)K(u)}{a_0(\alpha)a_2(\alpha) - a_1^2(\alpha)} I(-\alpha < u < 1).$$

```
K = function(u){
 0.75*(1-u^2)*ifelse(abs(u) <= 1, 1, 0)
}
a = function(l, alpha){
  integrate(function(u) u^l*K(u), lower = -alpha, upper = 1)$value
}
B = function(u, alpha){
  ((a(2, alpha)-a(1, alpha)*u)*K(u))/(a(0, alpha)*a(2, alpha)-a(1, alpha)^2)*ifelse(u <= -alpha, 0, 1)
}

alpha.v = c(0, 0.25, 0.5, 0.9)
plot(NA, xlim = c(-1, 1), ylim = c(-1, 8), xlab = "u", ylab = expression(B[alpha](u)))
for(i in 1:length(alpha.v)){
  f = function(u) B(u, alpha.v[i])
  curve(f, from = -1, to = 1, add = T, lty = i)
}
legend("topright", c(expression(paste(alpha, "= 0")),
                     expression(paste(alpha, "= 0.25")),
                     expression(paste(alpha, "= 0.5")),
                     expression(paste(alpha, "= 0.9"))), lty = 1:length(alpha), bty = "n")
```



Discussion:

When α is small, meaning that x is closer to the left boundary, the boundary kernel $B_\alpha(u)$ concentrates the weights around x and consequently mitigate the boundary bias. In contrast, as x moves away from the left boundary, the weights placed around x becomes smaller.

15.

(a)

The normalizing constant is derived as follows.

$$\begin{aligned}
&\int_{u_1^2+u_2^2\leq 1} (1-(u_1^2+u_2^2))du_1du_2 \\
&= \int_0^1 \int_0^{2\pi} r(1-r^2)d\theta dr \\
&= \frac{\pi}{2}
\end{aligned}$$

Hence, the normalizing constant is $\frac{\pi}{2}$.

(b)

The marginal pdf $f_{U_1}(u_1)$ is derived as follows.

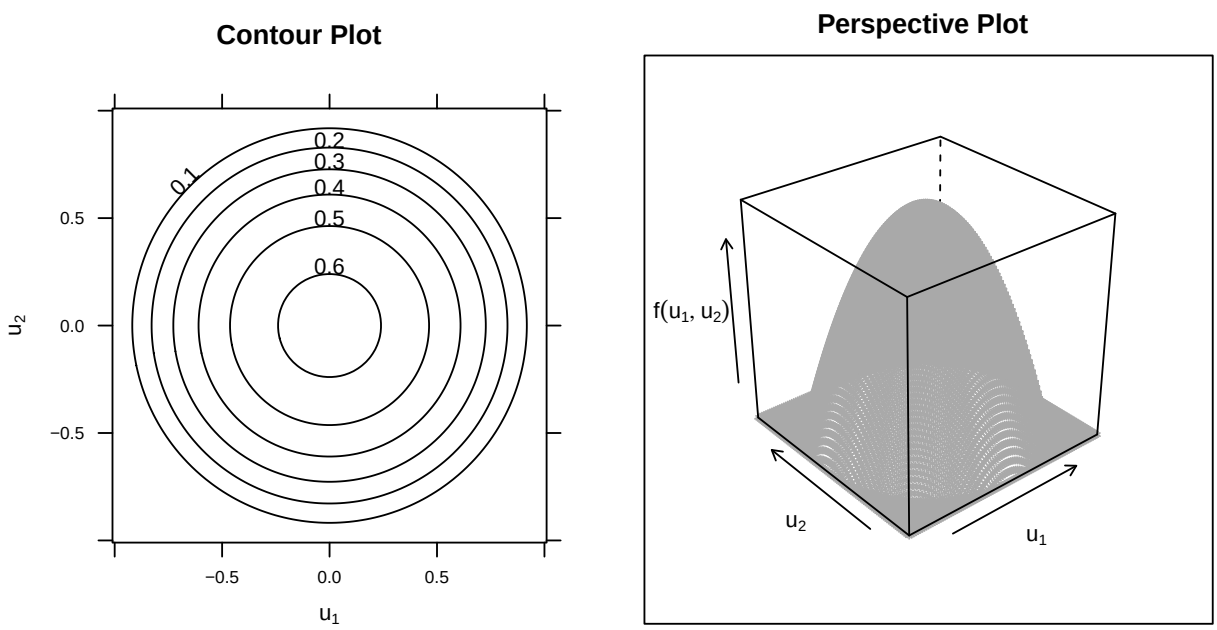
$$
\begin{aligned}
f_{U_1}(u_1) &= \int_{-\sqrt{1-u_1^2}}^{-\sqrt{1-u_1^2}} \frac{2}{\pi}(1-(u_1^2+u_2^2))du_2 \\
&= \frac{2}{\pi}\int_0^1 \int_0^{2\pi} r(1-r^2)d\theta dr \\
&= \frac{2}{\pi}[2(1-u_1^2)\sqrt{1-u_1^2} - \frac{2}{3}(1-u_1^2)^{3/2}] \\
&= \frac{8}{3\pi}(1-u_1^2)^{3/2}
\end{aligned}
$$

(c)

Below graphs the joint pdf $f(u_1, u_2)$ as quantile and perspective plots.

```
# Define the function
f <- function(u1, u2) {
  return ((2/pi) * (1 - (u1^2 + u2^2)))
}

# Generate grid points
u1_seq <- seq(-1, 1, length.out = 100)
u2_seq <- seq(-1, 1, length.out = 100)

# Create meshgrid
grid <- expand.grid(u1 = u1_seq, u2 = u2_seq)

# Compute function values
grid$z <- with(grid, f(u1, u2))

# Mask values outside the unit disk
grid$z[grid$u1^2 + grid$u2^2 > 1] <- 0

# Reshape for 3D plotting
Z_matrix <- matrix(grid$z, nrow = length(u1_seq), byrow = TRUE)

# contour_plot = contour(u1_seq, u2_seq, Z_matrix,
#                        xlab = expression(u[1]), ylab = expression(u[2]), main = "Contour Plot")
contour_plot <- contourplot(z ~ u1 * u2, data = grid,
                            xlab = expression(u[1]),
                            ylab = expression(u[2]),
                            main = "Contour Plot",
                            aspect = 1,
                            par.settings = list(fontsize = list(text = 9)))

# Generate the 3D cloud plot
persp_plot = cloud(z ~ u1 * u2, data = grid,
                   screen = list(z = 40, x = -60),  # Adjust viewing angle
                   col = "darkgrey",
                   xlab = expression(u[1]),  # X-axis label with subscript
                   ylab = expression(u[2]),  # Y-axis label with subscript
                   zlab = expression(f(u[1], u[2])),  # Z-axis label
```

```
                    main = "Perspective Plot",
                    aspect = c(1, 1),
                    par.settings = list(fontsize = list(text = 9)))

contour_grob <- grid.grabExpr(print(contour_plot))
persp_grob <- grid.grabExpr(print(persp_plot))

# ---- 4. Arrange Both Plots Side by Side ----
grid.arrange(contour_grob, persp_grob, ncol = 2)
```



16.

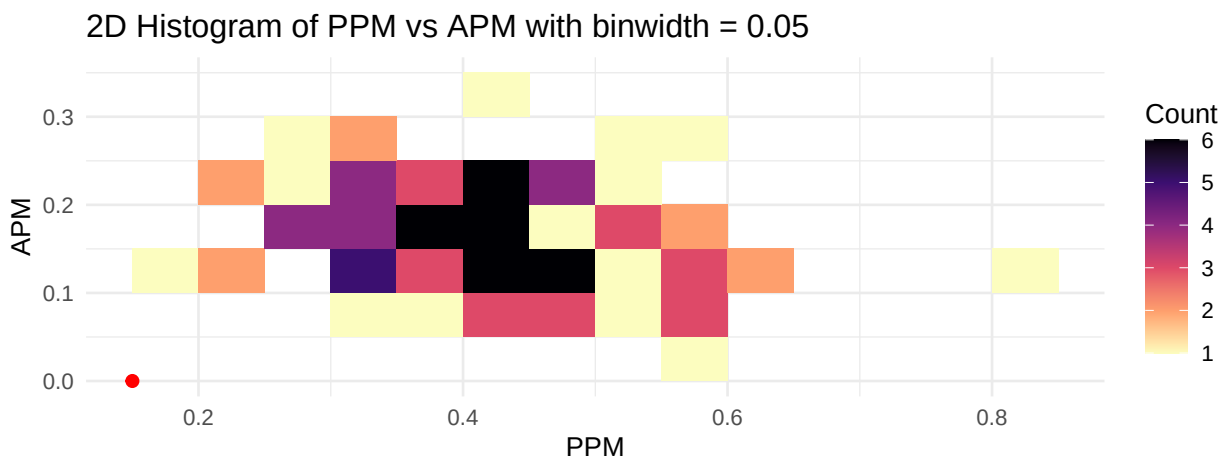
For the 2D histogram of PPM vs APM, I specified three bandwidths for both the x-axis and y-axis: 0.05, 0.1, 0.15, and the corresponding anchor points are: $(0.15, 0)$, $(0.1, 0)$, $(0.15, 0)$. For the 2D KDE of PPM vs APM, I specified the Gaussian kernel and specified three bandwidth matrices:

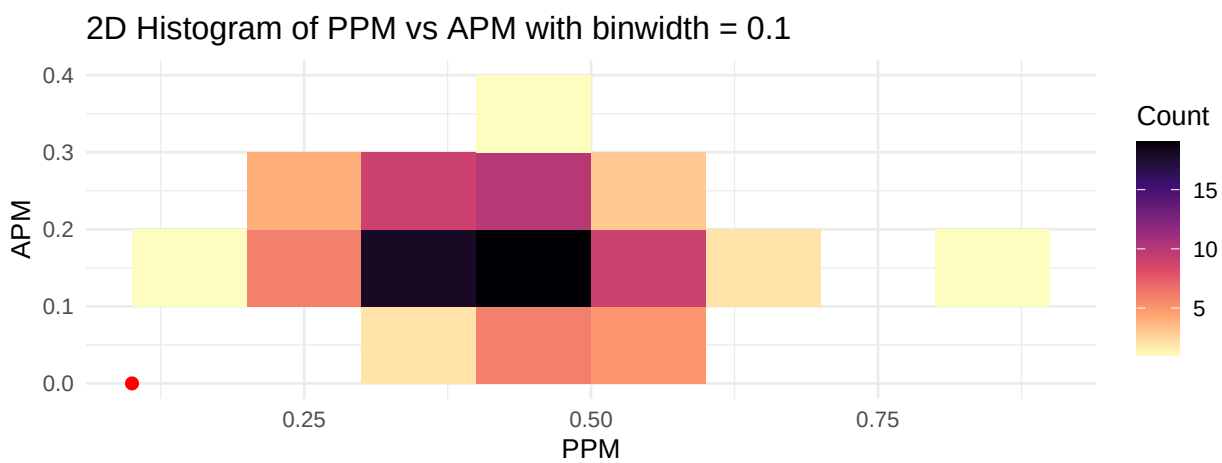
$$\begin{bmatrix} 0.05^2 & 0 \\ 0 & 0.05^2 \end{bmatrix}, \quad \begin{bmatrix} 0.1^2 & 0 \\ 0 & 0.1^2 \end{bmatrix}, \quad \begin{bmatrix} 0.15^2 & 0 \\ 0 & 0.15^2 \end{bmatrix}.$$

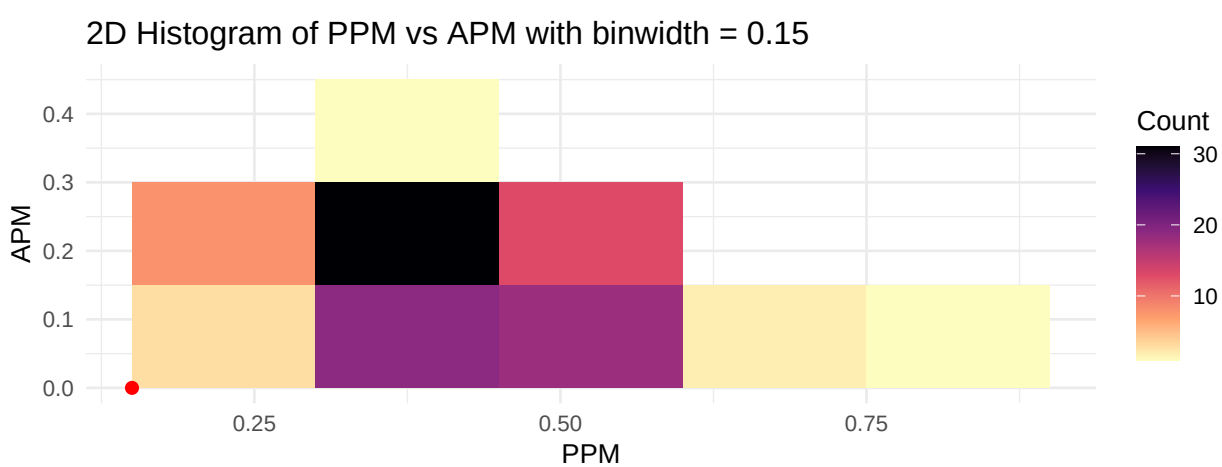
The graphs are shown below.

```
data = read.table("baskball.tab", sep = "\t", header = TRUE, fill = TRUE, quote = "", comment.char = ""
```

```r
hist_2d = function(bin_width){
  min_x = min(data$PPM)
  min_y = min(data$APM)
  ggplot(data, aes(x = PPM, y = APM)) +
  geom_bin2d(binwidth = c(bin_width, bin_width)) +  # Set bin width
  scale_fill_viridis_c(option = "magma", direction = -1) +  # Color gradient
  geom_point(aes(x = min_x, y = min_y), color = "red", size = 3) +
  theme_minimal() +
  labs(title = paste0("2D Histogram of PPM vs APM with bandwidth  = ", bin_width),
       x = "PPM", y = "APM", fill = "Count")
}

hist_2d = function(bin_width) {
  p <- ggplot(data, aes(x = PPM, y = APM)) +
    geom_bin2d(binwidth = c(bin_width, bin_width)) +  # Set bin width
    scale_fill_viridis_c(option = "magma", direction = -1) +  # Color gradient
    theme_minimal() +
    labs(title = paste0("2D Histogram of PPM vs APM with binwidth = ", bin_width),
         x = "PPM", y = "APM", fill = "Count")

  # Extract computed data
  bin_data <- ggplot_build(p)$data[[1]]

  # Bottom-left anchor points are just the xmin and ymin values
  anchor <- data.frame(
    anchor_x = min(bin_data$xmin),
    anchor_y = min(bin_data$ymin)
  )

  # Overlay the first anchor point on the histogram
  p = p + geom_point(data = anchor, aes(x = anchor_x, y = anchor_y),
                 color = "red", size = 2)

  return(list(p = p, a = anchor))
}

p1 = hist_2d(0.05)
p2 = hist_2d(0.1)
p3 = hist_2d(0.15)

grid.arrange(p1$p,
             p2$p,
             p3$p, nrow = 3)
```



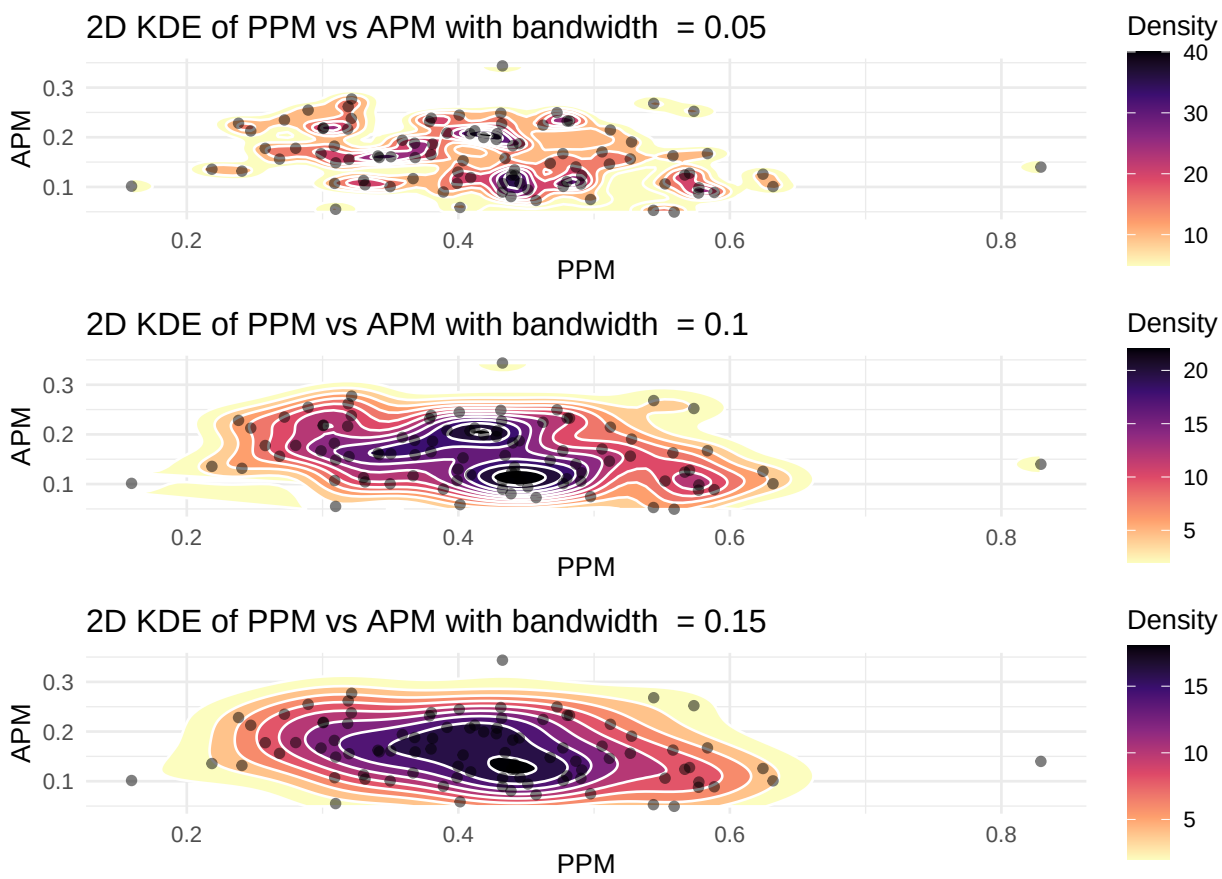





```
kde_2d = function(bin_width){
  ggplot(data, aes(x=PPM, y=APM) ) +
  stat_density_2d(h = bin_width, aes(fill = ..level..), geom = "polygon", colour="white") +
    scale_fill_viridis_c(option = "magma", direction = -1) +
  geom_point(alpha = 0.5) +
  theme_minimal() +
  labs(title = paste0("2D KDE of PPM vs APM with bandwidth  = ", bin_width),
       x = "PPM", y = "APM", fill = "Density")
```

```
}
grid.arrange(kde_2d(0.05),
             kde_2d(0.1),
             kde_2d(0.15), nrow = 3)
```

```
## Warning: The dot-dot notation (`..level..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(level)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Discussion:

As observed, as the bandwidth on each axis decreases, the colors in the 2D histogram become more uneven, and the contours of the density estimates appear more wiggly. Notably, when the bandwidth is set to 0.05, the density plot reveals a multimodal structure. In contrast, at a bandwidth of 0.1, the density plot exhibits a bimodal structure, while at 0.15, it smooths into a unimodal distribution.

Upon closer inspection of the data points, I suggest that a bandwidth of 0.1 may provide the best fit, as 0.05 results in an overly complex representation, while 0.15 oversmooths the data, potentially obscuring meaningful patterns in the distribution.

17.

1 Source Code

```
knitr::opts_chunk$set(echo = TRUE, fig.width = 7, fig.height = 5)
library(kableExtra)
library(tidyverse)
library(knitr)
library(plot3D)
library(lattice)
library(ggplot2)
library(gridExtra)
library(grid)
library(reshape2)
library(MASS)
library(viridis)
library(ks)
K = function(u){
 0.75*(1-u^2)*ifelse(abs(u) <= 1, 1, 0)
}
a = function(l, alpha){
  integrate(function(u) u^l*K(u), lower = -alpha, upper = 1)$value
}
B = function(u, alpha){
  ((a(2, alpha)-a(1, alpha)*u)*K(u))/(a(0, alpha)*a(2, alpha)-a(1, alpha)^2)*ifelse(u <= -alpha, 0, 1)
}

alpha.v = c(0, 0.25, 0.5, 0.9)
plot(NA, xlim = c(-1, 1), ylim = c(-1, 8), xlab = "u", ylab = expression(B[alpha](u)))
for(i in 1:length(alpha.v)){
  f = function(u) B(u, alpha.v[i])
  curve(f, from = -1, to = 1, add = T, lty = i)
}
legend("topright", c(expression(paste(alpha, "= 0")),
                     expression(paste(alpha, "= 0.25")),
                     expression(paste(alpha, "= 0.5")),
                     expression(paste(alpha, "= 0.9"))), lty = 1:length(alpha), bty = "n")
# Define the function
f <- function(u1, u2) {
  return ((2/pi) * (1 - (u1^2 + u2^2)))
}

# Generate grid points
u1_seq <- seq(-1, 1, length.out = 100)
u2_seq <- seq(-1, 1, length.out = 100)

# Create meshgrid
grid <- expand.grid(u1 = u1_seq, u2 = u2_seq)

# Compute function values
grid$z <- with(grid, f(u1, u2))

# Mask values outside the unit disk
grid$z[grid$u1^2 + grid$u2^2 > 1] <- 0
```

```r
# Reshape for 3D plotting
Z_matrix <- matrix(grid$z, nrow = length(u1_seq), byrow = TRUE)

# contour_plot = contour(u1_seq, u2_seq, Z_matrix,
#                        xlab = expression(u[1]), ylab = expression(u[2]), main = "Contour Plot")
contour_plot <- contourplot(z ~ u1 * u2, data = grid,
                            xlab = expression(u[1]),
                            ylab = expression(u[2]),
                            main = "Contour Plot",
                            aspect = 1,
                            par.settings = list(fontsize = list(text = 9)))

# Generate the 3D cloud plot
persp_plot = cloud(z ~ u1 * u2, data = grid,
                   screen = list(z = 40, x = -60),  # Adjust viewing angle
                   col = "darkgrey",
                   xlab = expression(u[1]),  # X-axis label with subscript
                   ylab = expression(u[2]),  # Y-axis label with subscript
                   zlab = expression(f(u[1], u[2])),  # Z-axis label
                   main = "Perspective Plot",
                   aspect = c(1, 1),
                   par.settings = list(fontsize = list(text = 9)))

contour_grob <- grid.grabExpr(print(contour_plot))
persp_grob <- grid.grabExpr(print(persp_plot))

# ---- 4. Arrange Both Plots Side by Side ----
grid.arrange(contour_grob, persp_grob, ncol = 2)
data = read.table("baskball.tab", sep = "\t", header = TRUE, fill = TRUE, quote = "", comment.char = ""
hist_2d = function(bin_width){
  min_x = min(data$PPM)
  min_y = min(data$APM)
  ggplot(data, aes(x = PPM, y = APM)) +
  geom_bin2d(binwidth = c(bin_width, bin_width)) +  # Set bin width
  scale_fill_viridis_c(option = "magma", direction = -1) +  # Color gradient
  geom_point(aes(x = min_x, y = min_y), color = "red", size = 3) +
  theme_minimal() +
  labs(title = paste0("2D Histogram of PPM vs APM with bandwidth  = ", bin_width),
       x = "PPM", y = "APM", fill = "Count")
}

hist_2d = function(bin_width) {
  p <- ggplot(data, aes(x = PPM, y = APM)) +
    geom_bin2d(binwidth = c(bin_width, bin_width)) +  # Set bin width
    scale_fill_viridis_c(option = "magma", direction = -1) +  # Color gradient
    theme_minimal() +
    labs(title = paste0("2D Histogram of PPM vs APM with binwidth = ", bin_width),
         x = "PPM", y = "APM", fill = "Count")

  # Extract computed data
  bin_data <- ggplot_build(p)$data[[1]]

  # Bottom-left anchor points are just the xmin and ymin values
```

```r
  anchor <- data.frame(
    anchor_x = min(bin_data$xmin),
    anchor_y = min(bin_data$ymin)
  )

  # Overlay the first anchor point on the histogram
  p = p + geom_point(data = anchor, aes(x = anchor_x, y = anchor_y),
                 color = "red", size = 2)

  return(list(p = p, a = anchor))
}

p1 = hist_2d(0.05)
p2 = hist_2d(0.1)
p3 = hist_2d(0.15)

grid.arrange(p1$p,
             p2$p,
             p3$p, nrow = 3)
kde_2d = function(bin_width){
  ggplot(data, aes(x=PPM, y=APM) ) +
  stat_density_2d(h = bin_width, aes(fill = ..level..), geom = "polygon", colour="white") +
    scale_fill_viridis_c(option = "magma", direction = -1) +
  geom_point(alpha = 0.5) +
  theme_minimal() +
  labs(title = paste0("2D KDE of PPM vs APM with bandwidth  = ", bin_width),
       x = "PPM", y = "APM", fill = "Density")
}
grid.arrange(kde_2d(0.05),
             kde_2d(0.1),
             kde_2d(0.15), nrow = 3)
```